Threat Modeling – Phase II

James Markijohn & Samaa Hediya

Project Overview

This project's goal was to build a web application that is capable of tracking users as they move through an art gallery. The gallery itself is a series of rooms with adjacencies to one another, which users can physically move through. The application tracks these movements in a database, along with each room's information, the user's account data, and security logs. The accounts for each user have a role associated with them (guest, employee, or administrator), which defines the level of access that the user has to this system. This role-based access control ensures that standard users cannot access sensitive data in the database, while privileged users can do so.

This application consists of four containers. The backend containers hold Django (most of the backend, including the APIs), PostgreSQL (for the database), and a backup container that takes regular encrypted backups of the database (using pg_dump with age for encryption). The fourth container is the Caddy frontend container, which is the only container that is exposed. All interactions with the backend containers must first pass through the Caddy container's reverse proxy (see Figure 1 for a visualization of this). Technologies used in this application include React and raw HTML/CSS for the frontend, Django and Python for the backend, PostgreSQL 18, Caddy, and Docker.

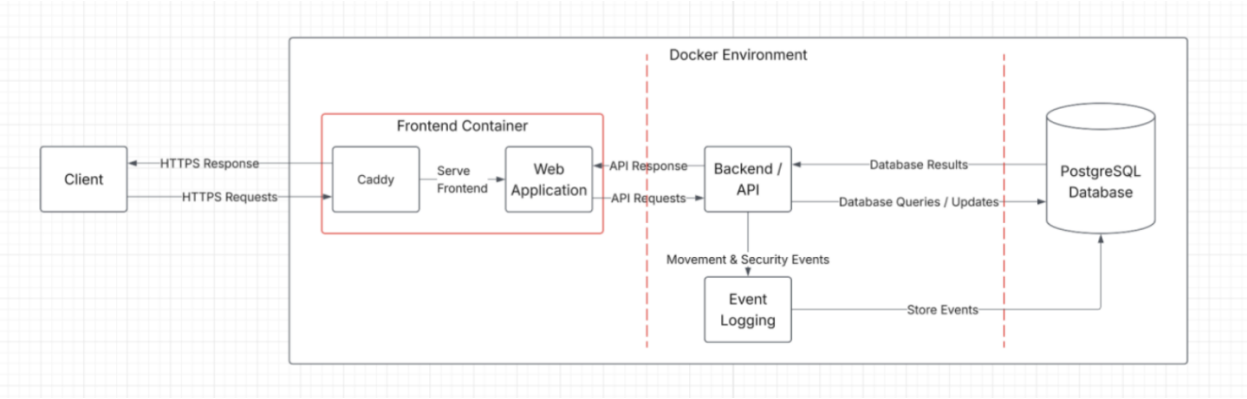


Figure 1. The diagram of the frontend-backend relationship, with trust boundaries indicated in red.

Assets

This web application handles several critical assets. The first asset is, generally speaking, the data stored by the application. In terms of personal data, this includes PII and contact information such as full names and phone numbers (which are optionally entered for each user) and email addresses (which are mandatory for account creation). There is also location data stored by this application, in the form of the current room that a user is in. The movement events table of the database stores historical location data. Other data assets include password hashes and the TOTP secrets used for MFA.

Other assets pertain to the application itself. These include the secrets stored in the .env file, which will be unique for each instance of the application. This file contains the secret keys used for Django, field-level encryption of the PostgreSQL databases, and the password for the database. A separate file will also store the private key for the age-encrypted database backup. If these keys were stolen by a bad actor, that attacker would gain access to the backend, the production database, and/or the backed-up data.

System components that could be considered individual assets include the database dump file, the production database, the Caddy reverse proxy, and (generally) the backend. Specifically, within the backend, each API endpoint group could be called an asset. See the "System Decomposition" section for details pertaining to these groups. In the worst-case scenario, an attacker with unauthorized access to an administrative endpoint could be able to view and modify sensitive data within most tables of the database.

External assets of this application include the authenticator app used by the end users, the base images used in the Docker containers (such as the PostgreSQL 18 image), and the packages used by the software. These packages include various Python libraries and packages managed by npm. Should one of the libraries, packages, or images be compromised in some way (such as in a supply chain attack), this could introduce vulnerabilities into the application. Additionally, the authenticator app is of particular concern for end user accounts, as a disreputable app could steal TOTPs.

Threat Actors

There are several types of threat actors that may attempt to access this system. Internal threat actors include administrators or employees who decide to abuse their privileged accounts by tampering with the databases. This would allow them to promote guest accounts to a higher level of privilege, delete records of movement events, and modify account information (such as names and numbers). These attacks could also be performed with a compromised employee or administrator account (which is why password requirements are enforced, along with MFA). Additionally, users with guest accounts may also attempt to elevate their privileges in order to view or modify sensitive data. A simpler attack would be to attempt to access administrator API endpoints or dashboards without escalating their privileges.

External threat actors include attackers without an account. Rather than creating an account and being provided with the default "guest" role, an attacker may instead use a credential stuffing attack to compromise a privileged account. Alternatively, at the login and signup pages, an unauthenticated attacker may attempt an injection attack in order to modify or extract data stored in the database. In this project, the Break It team will also be an external threat actor, akin to penetration testers.

System Decomposition

The "Overview" section provided a high-level discussion of the system's architecture (see Figure 1 in that section). To summarize, there are four containers in this system: three backend containers (Django, PostgreSQL, and a database backup container) and one frontend container (Caddy and the React frontend).

In the system's backend, there are four groups of APIs: auth, gallery, admin, and audit. The auth API handles authentication (multi-factor) and account management, along with session termination and CSRF cookies. The admin API is a privileged API that is used for administrative actions, such as modifying databases and creating other privileged accounts. This API is only made available to administrator accounts. It allows modification of database tables, resetting of MFA for any account, adjustment of room adjacencies, and account deletion. The gallery API's endpoints are primarily restricted to employees and administrators so that they can see the location of each user, manually record a movement event, and view movement events. Administrators may also delete movement events (the use case here is to allow deletion of duplicate events or events added to the table in error), and guest users may view their own current room only. Finally, the audit API simply allows administrators to view the security log table of the database; it does not permit modification of that log.

This application's PostgreSQL database contains several tables, which are visualized in Figure 2 below. The accounts table will have a many-to-one relationship with the gallery rooms table (many accounts may be present in a single room), a one-to-many relationship with the security logs (one account may generate many logs), and a one-to-many relationship with the movement events table (one account may move many times). The accounts table's password field contains an Argon2id hash value, and the name, phone number, and TOTP (MFA) secret are encrypted with field-level encryption. The security log table's details field will be encrypted as well. The gallery rooms table will have a many-to-many relationship with the movement table (a movement will be associated with exactly two rooms, and many movements can be associated with the same room), and a one-to-many relationship with the adjacent rooms table. The adjacent rooms table simply stores adjacency data for each room in gallery rooms; this was done for development convenience. The entity-relationship diagram in Figure 2 does not show the rate limit hits database, which is a small table that tracks recent requests, their timestamps, and the IP making the request for the purpose of rate limiting; it is not related to any other table. The database is dumped and backed up using pg_dump, then encrypted with age using a unique key.

Figure 3 shows this application's data flow diagram, along with the trust boundaries established by the application. This diagram, like Figure 1, indicates that requests to the backend must first pass through the Caddy reverse proxy in the frontend container; direct backend access is not permitted. A trust boundary is established between the users and the backend application due to this feature. The second trust boundary lies between the backend and the databases, the latter of which may only be accessed through API calls. The trust boundary is enforced by the

API endpoints, which are only made available to user accounts with the appropriate role applied. It is also worth noting in this diagram that the security log is append-only. That is, events occur and trigger a security log to be written to that table, but there is no API endpoint or administrative function that allows an account to delete records from that table. This allows auditing of significant events, such as authentication attempts, movement into unauthorized areas, and querying of the database tables.

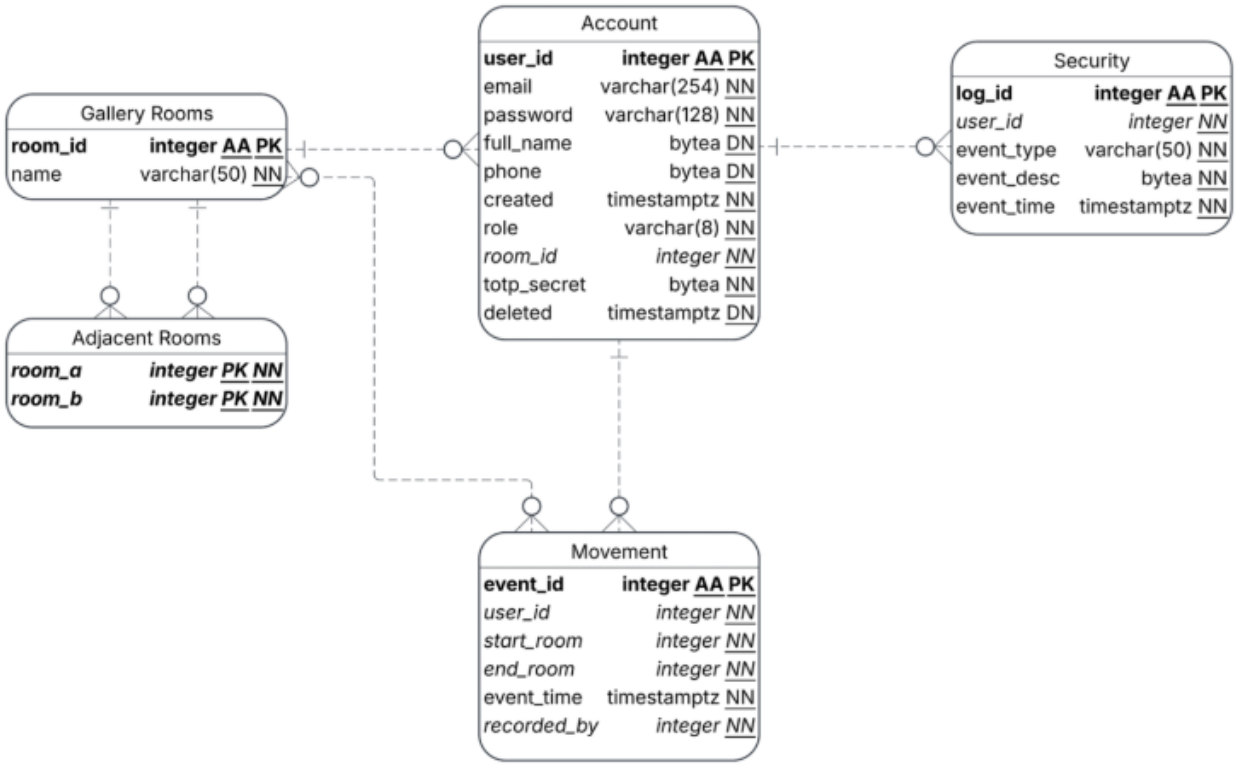


Figure 2. The entity-relationship diagram for this application's database.

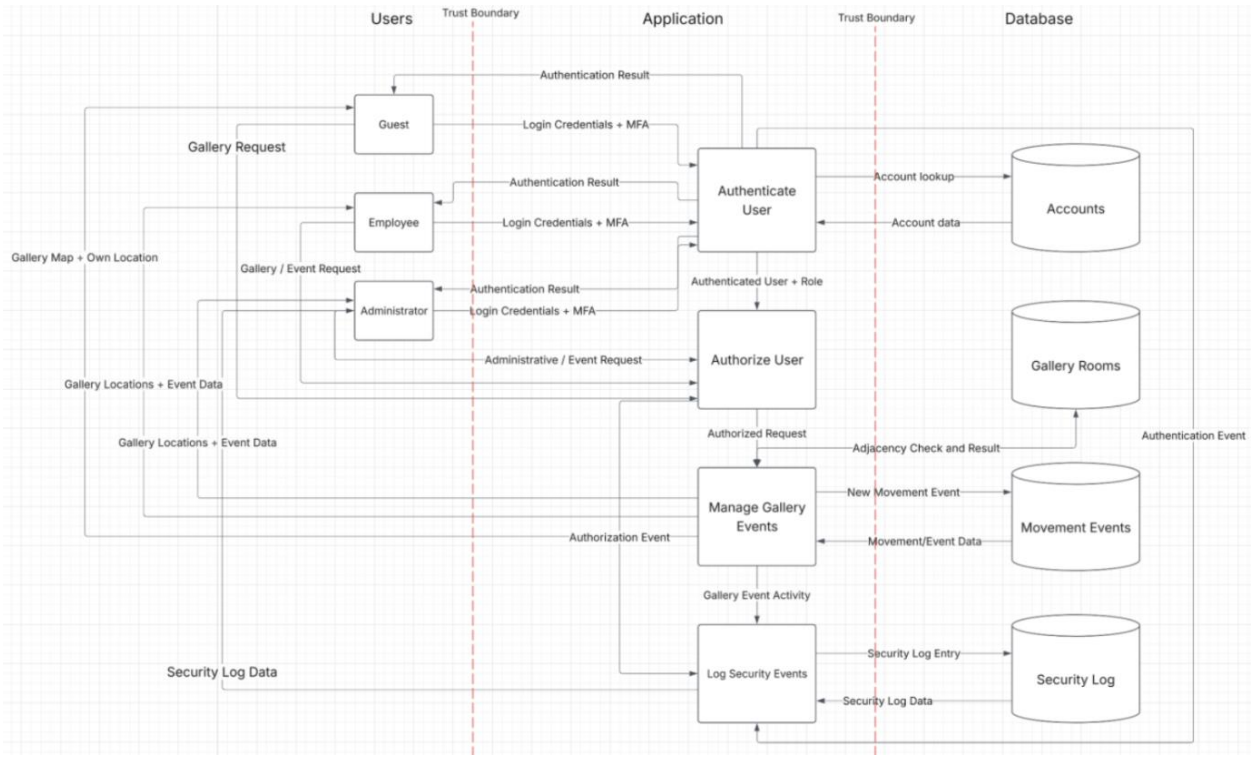


Figure 3. The data flow diagram for the application, with trust boundaries, backend systems, and database tables.

STRIDE

Table 1 below shows the STRIDE information developed during the requirements phase of the development lifecycle. This information was also provided on this project's design document, and was frequently referenced throughout development. More information on how this table was used during development can be found in the "Threat Modeling Documentation" section of this report.

Spoofing	• Impersonation of real individuals (such as administrators or employees) during guest account creation. o Cannot verify the identity of users who create guest accounts without ID verification methods, but guest accounts will have only the minimum privileges applied, so impersonation will not be an effective threat vector. • Attackers may attempt to insert forged records into the movement event and security log database tables, including the identity of the account that is associated with those events. o Databases can only be modified through queries sent through a backend API, which will not be made accessible to guest users; attackers would need the administrator role to perform this attack. Additionally, administrators can edit databases to remove any forged records if they are detected.
Tampering	• A user may attempt to modify data sent to the application, such as changing movement information to record an invalid location or movement. o Databases may only be accessed via a backend API, which will not be accessible to guest users. Parameterized queries will also mitigate the chances of unauthorized modifications of the movement event database. A compromised employee or administrator account can bypass these protections.
Repudiation	• A user may perform an unauthorized action within the application, or attempt to perform a dangerous operation, then claim that they did not do so. o The security log will contain any events that were triggered by specific user actions, which will allow administrators to provide non-repudiation. This also applies to the application's movement event database, which will contain all the movements of users; if a guest enters a zone that is out-of-bounds to them, this database will show that.

Information Disclosure	• Unauthorized database access could result in the entire contents of the application's databases being read and/or exfiltrated. o This can be mitigated through preventing SQL injection through the use of parameterized queries, enforcing authorization based on role-based access for user accounts, and enforcing MFA to prevent instances of compromised accounts with higher levels of privilege. A trust boundary exists between the application and the databases, facilitated through the backend API. • The location data for each guest and employee in the gallery will be visible on the map in the web interface of the employee or administrator accounts. Attackers may gain access to these maps if they compromise a privileged accounts. o MFA helps mitigate the instances of compromised passwords, though if an attacker gained access to a privileged account, there would be no mechanism to prevent them from viewing the map with the location data included.
Denial of Service	• An attacker may send a large number of requests to the application in a short period of time, causing the server to become overloaded and preventing legitimate users from accessing the application.
Elevation of Privilege	• A user may attempt to gain permissions that belong to a higher-level role (such as a guest gaining employee privileges, or an employee gaining administrator privileges). o The role of the user will be inserted in the accounts table of the database and cannot be changed by anyone without administrator-level privileges. Guest user accounts should not be able to change their roles (or directly access the database at all, for that matter). o An attacker may compromise an account to gain elevated privileges, though MFA is implemented to mitigate this type of attack.

Table 1. STRIDE classification of threats based on analysis of the application during the requirements phase.

Threat Analysis and Prioritization

The risk matrix below shows the severity of the threats that are relevant to our system, categorized based on their likelihood and impact. These are qualitative and non-comprehensive definitions but still provided a framework which was used to categorize and rate the threats identified.

Here, the definitions for likelihood are divided into low (attacker requires a privileged account, must compromise dependencies, precise timing), moderate (attacker requires a guest account, or no account but specific timing and precise steps), and high (attacker requires no account, can exploit a vulnerability at any time).

Similarly, the definitions for impact are divided into low (minor information such as email address data is leaked, brief denial of service for small group of users), moderate (compromising a guest user account and accessing that user's data, adding entries to database tables, widespread denial of service for a large group of users), and high (mass exfiltration and exposure of raw data from the database, compromise of an administrator's account, tampering with the security log).

Based on analysis of Table 2 below, the most critical threats facing our system were deemed to be injection attacks, brute-force attacks that result in compromised administrator accounts, XSS and CSRF attacks, session hijacking, and privilege elevation of guest accounts. Threats involving user impersonation, appending fake movement events to the movement events table of the database, and attempts to expose registered email addresses were less significant in comparison. Abuse of the rate limit system, DDoS attacks, and reading of current location data were examples of moderate threats.

Likelihood x Impact	Low (1)	Moderate (2)	High (3)
Low (1)			Impersonation of real individuals (3) Attempting to determine which emails are registered (3)
Moderate (2)	Inserting fake movement events into database (2)	Deliberately locking accounts by abusing rate limiting system (4) Guests reading current locations of other guests (4)	DDoS attack triggered by request flood (6)
High (3)	Guests reading movement events table (3) Promotion of guest accounts to administrator accounts (3) Gaining physical access to an administrator's device or the server hosting the application (3)	MFA-based brute force attacks (6) XSS attacks based on user input for name and phone number fields (6) CSRF attacks that make changes to system as administrator (6) Session hijacking (6)	SQL injections (9) Password-based brute force attacks (9)

Table 2. Risk matrix showing severity of threats, classified by likelihood x impact.

Mitigation Strategies

Given the threats that were identified by STRIDE analysis (Table 1) and classified in the risk matrix (Table 2), the following mitigation strategies were implemented. Table 3 below shows each of the aforementioned threats alongside a mitigation strategy and the reasoning behind that particular strategy. Whenever a threat could not be entirely eliminated, the table acknowledges this and explains why the mitigation strategy used was considered sufficient.

Threat (risk score)	Mitigation	Rationale
Inserting fake movement events into the database (2)	Administrators can delete events from the movement events database	An attacker may wish to enter fake movement events in the database to obscure real movement. To combat this threat, administrators were given the ability to manually delete movement events from the database. This functionality introduces its own set of threats, but it sufficiently mitigates this one and allows administrators to act against events added to the database due to a bug or data corruption.
Impersonation of real individuals (3)	No mitigation was deemed necessary for this threat	The role-based access control used in this system does not make use of the "full name" field in a user's account, so impersonating an administrator by creating a guest account with their full name will not be an effective threat vector. The best way to mitigate this threat is through ID verification of new users, which was not feasible for this project.
Attempting to determine which emails are registered (3)	Rate limiting and vague error messages	Attackers may wish to produce a list of all registered email addresses in this application by brute-forcing their way through a list of addresses and using information from error codes to determine which are valid. The login error messages do not specify if the email address or password was incorrect after a failed login, and login attempts are significantly rate limited such that a brute force attack of this nature would be infeasible.
Guests reading movement events table (3)	Role-based access control, restricted access to API endpoints, different web interfaces for each role	To mitigate the threat where guest users attempt to read historical movement data for other users, the API endpoints that allow access to the movement events table were limited to only employees and administrators. That is, accounts with the

		guest role will not be able to use this endpoint, while those with employee or admin as their role can. Additionally, each type of user will have a different web interface, meaning guest users will not even see the same frontend that allows employees and administrators to view and query the movement events table.
Promotion of guest accounts to administrator accounts (3)	Role-based access control, restricted access to API endpoints, different web interfaces for each role	The strategies used here are similar to those in the previous strategy. Since users can be promoted to a higher role only by administrators in the admin-only web interface, access to this frontend was restricted to admin accounts only. This also applies to the API endpoint used to promote the user in the backend.
Gaining physical access to an administrator's device or the server hosting the application (3)	Data encryption, password hashing	The physical security of the hardware used to host this application was beyond the scope of this project, but it is worth stating that an attacker who gains physical access to the disk or device hosting the application could potentially steal it, along with its data. Passwords are hashed to prevent passwords from leaking in this manner. While data is encrypted, if the keys are stored on the same device as the application, this will not be an effective mitigation. An attacker who gains access to an admin's device would effectively gain admin access to the system, so in a real production environment, these systems would need to be kept in staff-only areas of the gallery.
Deliberately locking accounts by abusing rate limiting system (4)	Client-based rate limiting	To make it more difficult for an attacker to deny legitimate users access to their account, rate limiting for login attempts is determined based on the IP address sending the requests rather than the account that is the subject of the requests. That is, if an attacker floods the login system with requests, their IP will be blocked, while a legitimate user's will not. If the attacker is on the same network as a legitimate user, this mitigation strategy will be bypassed.

Guests reading current locations of other guests (4)	Role-based access control, restricted access to API endpoints, different web interfaces for each role	Again, API endpoints and web interfaces that expose this data are restricted to accounts with employee and admin roles only. See the row for "Guests reading movement events table" for details.
DDoS attack triggered by request flood (6)	Rate limiting, minimal service exposure	The rate limiter in this system was implemented to prevent DDoS attacks of this nature. Attackers will be prevented from sending requests for several minutes if an anomalous number of requests is detected from their IP address. Furthermore, since the backend services are placed behind the Caddy reverse proxy, they will not be directly hit by a flood of requests. This should prevent systems such as the Django backend and PostgreSQL database from being overloaded.
MFA-based brute force attacks (6)	Rate limiting, time limits for MFA codes	The TOTPs used for MFA are generated in accordance with RFC 6238. A secret key is created for an account upon creation, which is displayed to the user in plaintext and as a QR code, then entered into their authenticator app. The codes generated are valid only for thirty seconds, meaning a brute force attack would need to iterate through all 6-digit possibilities in this time. As this is computationally feasible (10^6 checks), rate limits are implemented to drastically slow down the rate at which attackers can attempt code combinations. This threat also assumes that the attacker has obtained an account's password (see "Password-based brute force attacks").
XSS attacks based on user input for name and phone number fields (6)	React escaping, CSP	Since users may provide input for their full name and phone number which will then be displayed to administrators in the web interface, React escapes these values so that they cannot be interpreted as code by the administrator's browser. A Content-Security-Policy (CSP) header was also configured. This application's CSP explicitly prevents external script files, stylesheets, and images from being

		executed or used, and rejects other external resources by default.
CSRF attacks that make changes to the system as an administrator (6)	CSRF tokens, SameSite=Strict setting	The application protects against CSRF attacks by generating a CSRF token server-side and supplying that to the client. Additionally, the SameSite=Strict header was configured for this application, which forbids cookies from being sent on cross-site requests.
Session hijacking (6)	Secure headers, session revocation on account changes	The Set-Cookie header was configured with HttpOnly to restrict how cookies are accessed and Secure so that the cookie is only sent over HTTPS connections. Sessions are also revoked when changes are made to an account by an administrator, such as a password reset or role change.
SQL injections (9)	Django ORM for queries to database, no direct database access	Since a PostgreSQL database is used in this application, SQL injection attacks are a substantial threat. When a user provides any text input into this application, that data will not be sent directly to the database, which would allow SQL injection attacks to occur. Instead, the input is passed to Django's ORM, which natively protects against SQL injection by automatically generating parameterized queries. ORM is used for every field where input is expected from the user.
Password-based brute force attacks (9)	Rate limiting, password requirements, MFA	Credentials obtained with brute force methods can lead to a compromised administrator account in the worst case. MFA was implemented to ensure that, even if a password was obtained, attackers could not easily gain access to an account. Password requirements were also stipulated to prevent users from using weak or common passwords that would be easily guessed in a brute force attack. Rate limiting is also used to limit the efficacy of brute force attacks, as attackers can make very few requests within a short time span before they are restricted from login attempts for several minutes.

Threat Modeling Documentation

Threat modeling was primarily performed during the first phase of this project, when the requirements were being listed. Along with the functional requirements of the web application, the security requirements were described with respect to the planned architecture of the application. Once these general security requirements were described, the actual threat modeling phase began.

This project's threat modeling relied on two tools: the STRIDE model, and the Microsoft Threat Modeling Tool. The STRIDE model was helpful for identifying areas in our system where common types of attacks could be initiated. Using this framework, a list of potential attacks that bad actors could carry out against our system was created, along with an abstract list of potential mitigation techniques for each attack. For a more specific set of vulnerabilities, the Microsoft Threat Modeling Tool was used. A mockup of our system's architecture was built within the tool, which then generated a list of vulnerabilities that could be analyzed and prioritized during development of the application.

During the initial phases of development, the Microsoft Threat Modeling Tool's analysis was frequently referenced. This was done so that the potential vulnerabilities identified by the tool could be mitigated during the development process, rather than after (in accordance with the "push left" design principle). As major development milestones were met, the STRIDE model was reviewed to verify that the abstract mitigation techniques were gradually being satisfied by the mitigation strategies used to protect the system against the Microsoft Threat Modeling Tool's identified vulnerabilities. By the time the application's development was completed, another review was performed on the STRIDE and Microsoft Threat Modeling Tool materials to ensure that every identified vulnerability was mitigated as well as feasibly possible.

For an overview of the entire system, see the "Overview" section and the high-level diagram in Figure 1. Details of the database can be found in Figure 2. The "System Decomposition" section of this report provides a detailed breakdown of system components and their interactions, which is supplemented by the data flow diagram in Figure 3.